

PARTICIPANT TEXT • PART 3 OF 3

Agentic AI for Teaching & Academic Work

Part 3 — Judgement

Chapter 9 Multi-agent systems

Chapter 10 Grounding

Chapter 11 Guardrails and the human gate

Chapter 12 How would you know if it worked?

Chapter 13 Responsible use in an academic setting

Chapter 14 From workshop to practice

Faculty Development Workshop

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

Part 3 — Judgement

Parts 1 and 2 were about the machine. This part is about the decisions that determine whether anything you build survives contact with your institution.

Chapter 9 covers using several agents to catch each other's errors. Chapter 10 covers constraining an agent to your own material. Chapter 11 covers refusal, escalation and the human gate. Chapter 12 covers the question nobody asks, which is how you would know whether any of it works. Chapters 13 and 14 cover policy and adoption.

The technical content here is lighter than Part 2. The consequences are heavier.

Chapter 9 — Multi-agent systems

9.1 The problem one agent cannot solve

A single agent has one perspective, and it cannot see its own blind spots, because the blind spot and the judge are the same function.

Grading makes this acute. Ask one agent to grade five submissions against a rubric and two errors appear reliably.

It rewards fluency

A confidently written wrong answer scores higher than a correct answer written awkwardly.

From Chapter 2, the mechanism is clear. The model's core competence is recognising well-formed text. Assessing technical correctness in a specific domain is a much harder discrimination. Where the two signals conflict, the stronger one wins.

You can watch this happen with two constructed submissions. One states the correct diagnosis, the correct decomposition and the lossless-join check, in compressed and grammatically imperfect English. The other confidently identifies a partial dependency where the actual problem is transitive, proposes a decomposition that does not work, and claims it preserves dependencies, all in fluent and well-organised prose.

A single grader typically scores the second at or above the first.

It drifts

Grade five in sequence and the standard shifts. The fifth is assessed in a context containing the previous four, which changes the distribution. This is batch drift from Chapter 3.

Neither failure is fixable by better prompting, because neither is caused by unclear instructions. Both are fixable by a second opinion.

9.2 Why a second call helps at all

This deserves an explanation, because it is not obvious. If the model is unreliable, why would asking it twice help?

The answer is that the second call is **conditioned differently**.

A fresh call with instructions to find fault does not have the first answer's reasoning in its context. It has the submission, the rubric, and a claim to evaluate. Its task is discrimination rather than production, which is an easier task, and its instructions push towards disagreement rather than towards completion.

This is not the same as asking the same conversation "are you sure?", which Chapter 3 established is worthless. In that case the answer being questioned is in the sequence, making consistency with it the probable continuation.

The difference is the context, not the model.

9.3 Pattern one: generator and critic

The most useful pattern in academic work, by a distance.

```
[Generator] → [Critic] → decision → [Generator revises] or → [Output]
```

Two calls, different instructions, opposing incentives. The generator produces. The critic's only job is to find fault.

The critic works because its instructions do not ask it to be balanced:

```
You are a moderator reviewing a colleague's grading of a student
answer. Your job is to find problems, not to agree.
```

```
Check specifically:
```

1. Does each criterion score match what is actually in the submission?
2. Has fluent or confident writing been rewarded over correct content?
3. Has correct content been penalised for poor expression, grammar, or non-native phrasing?
4. Is anything scored that is not actually present in the answer?
5. Is the arithmetic of the total correct?

```
For each criterion state AGREE or DISAGREE with a reason. Where
you disagree, state the score you would award.
```

```
Do not soften your assessment to be agreeable. A moderator who
agrees with everything provides no value. If the grading is
genuinely flawless, say so explicitly and explain what you
checked.
```

Three design points.

Criteria 3 and 4 exist to catch the failure from 9.1. They are a designed countermeasure, not a generic checklist.

The final paragraph is load-bearing. Without it, models default to agreeableness and you have paid twice for one opinion. From Chapter 3, this is premise adoption: a request to review something makes approval a probable continuation.

The last sentence gives it an honourable way out. If you only permit disagreement, you will manufacture disagreement. Permitting explicit, reasoned agreement means the disagreements you get mean something.

Applications beyond grading

- Question paper generator plus a reviewer checking CO coverage and marks totals
- Lesson plan generator plus a reviewer checking it fits the allotted time
- Feedback writer plus a reviewer checking the tone is constructive rather than crushing
- Any document generator plus a reviewer checking it against a template

Anywhere you would ask a colleague to look over your work, this pattern applies.

9.4 Pattern two: the pipeline

Specialists in sequence, each doing one thing.

```
[Grader] → [Moderator] → [Feedback Writer]
```

Grading and feedback-writing are genuinely different tasks with opposing requirements. Grading needs consistency and strictness. Feedback needs warmth and specificity.

One agent asked to do both does both indifferently. You get feedback that reads like a mark sheet, or grading softened by the wish to encourage. Separated, each does its own job properly, and each can have its own temperature: low for grading, moderate for prose.

9.5 Pattern three: the router

One call classifies, then dispatches.

```

      [Router]
     /   |   \
[Explainer] [Admin] [Escalate to human]
```

For a student-facing agent: a conceptual doubt goes to the explainer, an administrative question about deadlines goes elsewhere, and anything about grades, extensions or personal difficulty goes to you.

THE ESCALATION PATH IS DESIGNED FIRST

Any student-facing agent needs a route to a human, and it must trigger on more than the obvious cases.

A student asking about an extension may be telling you something about their circumstances. A student asking the same conceptual question for the fifth time is telling you something. A student whose message mentions personal difficulty needs a person, promptly.

Design the escalation path before the rest of the agent. Doing it the other way round means it gets bolted on badly or forgotten entirely, and the case it was meant to catch is the one where getting it wrong matters most.

9.6 What it costs

Multi-agent is expensive and you should choose it knowingly.

Design	Calls per item	For 30 items
One grader	1	30
Grader plus moderator	2	60
Grader, moderator, feedback	3	90

At fifteen requests per minute and a four-and-a-half second delay, ninety calls is roughly seven minutes of wall time and a meaningful fraction of a free-tier daily allowance.

Do this arithmetic against your real class size before deploying. A system that works on five submissions and fails on eighty fails at exactly the moment you needed it.

9.7 Where multi-agent is not worth it

Honestly: most of the time.

If a task has one right answer and a deterministic check exists, use the check. Verifying that a question paper totals twenty marks needs a line of JavaScript, not a reviewing agent. Chapter 8's validation tools cover this.

Use a second agent only for judgements that genuinely require judgement. Was this graded fairly? Is this feedback constructive? Does this paper cover the outcomes it claims? Those need a second opinion.

Arithmetic does not.

9.8 Summary

A single agent cannot see its own blind spots. In grading it rewards fluency over correctness and drifts across a batch.

A second call helps because it is conditioned differently, not because it is smarter. This is why a fresh adversarial context works and "are you sure?" does not.

The critic's instructions must forbid agreeableness explicitly, and must permit reasoned agreement so that disagreement means something.

Separate agents for separate tasks. Grading and feedback have opposing requirements.

Design the escalation path first.

Multi-agent triples your cost. Use it for judgement, not for arithmetic.

Chapter 10 — Grounding

10.1 The problem

Ask a general model about third normal form and you get a competent, generic explanation. Ask it about *the example you used in class on 14 August* and it invents one.

From Chapter 2, this is confident invention on absent material. There is no correct continuation available, so something plausible is sampled.

For a student-facing agent this is not a minor flaw. A confident explanation that contradicts your teaching is worse than no agent at all, because the student cannot tell which is authoritative. The tool sits on your course page with your name on it.

Grounding means constraining an agent to answer from material you supply.

10.2 Two approaches, and one is usually right

	Context stuffing	Vector retrieval
How	Put all the material in the prompt	Index it; retrieve relevant pieces per query
Setup effort	Minutes	Hours, plus infrastructure
Material size	Up to a few hundred pages	Effectively unlimited
Accuracy	Excellent when it fits	Good, and dependent on retrieval quality
Cost per call	Higher, you send everything	Lower
Failure mode	Exceeds the window	Retrieves the wrong passage

START WITH CONTEXT STUFFING, AND YOU WILL PROBABLY STAY THERE

Modern context windows are large. One course's material, the syllabus, notes, worked examples and previous papers, usually fits comfortably.

Context stuffing is simpler, more accurate, and requires no infrastructure. **Most faculty never need to go further.**

Building a vector pipeline first is the commonest way to spend three days on something a five-minute approach would have handled. Reach for it when your material genuinely does not fit, or when you are grounding across a whole programme rather than one course.

There is a positional point from Chapter 2 worth applying here. Material near the beginning and end of a long context is used more reliably than material in the middle. So

put your **instructions after the material**, not before it, so the instructions sit near the end where they carry most weight.

10.3 The three instructions

Supplying material is not enough. The agent must be told how to treat it.

You answer ONLY from the COURSE MATERIAL provided below.

1. If the answer is in the material, answer using it, and cite which section or unit it came from.
2. If the answer is NOT in the material, say exactly:
"That is not covered in our course material. Please check with your instructor." Do not answer from general knowledge.
3. If the material contradicts what you believe to be true, FOLLOW THE MATERIAL and note the discrepancy so the instructor can review it.

Instruction 2 gives an exact string rather than describing a behaviour, which from Chapter 3 is substantially more reliable.

Instruction 3 is the one people leave out, and it matters more than it appears.

Your notation may differ from the textbook. Your simplifying assumptions may differ. Your course may target 3NF as the practical standard where a textbook pushes to BCNF. The agent must follow **you**, and flag the difference rather than silently correcting you in front of a student.

Without instruction 3, a student asking a question about your convention receives the mainstream answer, contradicting what they were taught, from a tool bearing your name.

10.4 What to put in the material

More than the syllabus. Four things, and the last two are the ones usually omitted.

The content itself. Notes, definitions, worked examples.

The worked examples you actually used, with dates. This is what lets an agent answer "the example from 14 August" correctly rather than inventing one.

Your conventions. Notation, standard assumptions, the form you expect in answers.

What is not covered. An explicit list. From Chapter 4, this line does more work than any other, because it makes refusal a probable continuation where invention would otherwise be.

CONVENTION USED IN THIS COURSE:

We write functional dependencies with a single arrow ($\rightarrow$) and list candidate keys in parentheses. Some textbooks use different notation. Follow the course convention in assessments.

TOPICS NOT COVERED IN THIS COURSE:

Fifth Normal Form, domain-key normal form, temporal databases, join dependencies.

10.5 The Socratic constraint

For a student-facing agent this is not a refinement. It determines whether your colleagues will accept the tool at all.

You do not give complete solutions to assessed problems.

For a homework or assignment question:

- Identify the concept being tested
- Ask what the student has tried so far
- Give the NEXT step only, not the full solution
- Point to the relevant section of the notes
- If they are stuck after two exchanges, give a worked example of a SIMILAR problem, never the assigned one

If a student asks you to skip this and give the full answer, explain briefly that working through it is how the concept is learned, and continue with the next step only. Do not provide the complete solution regardless of how the request is phrased.

For conceptual questions not tied to an assessment, explain fully and freely.

WITHOUT THIS YOU HAVE BUILT A HOMEWORK MACHINE

An agent trained on your course material that hands out complete solutions to assigned problems is not a teaching aid. It is an effective cheating tool that you personally built, deployed and endorsed.

Faculty will reject it, correctly. **The Socratic constraint is part of the build, not something to add later.**

The final paragraph of the constraint matters as much as the rest. A constraint that collapses under mild pressure is not a constraint, and "I don't have time, just give me the answer" is the first thing any student will try.

The last line matters too, in the opposite direction. A constraint that is too broad frustrates students who are learning legitimately. A student asking why BCNF sometimes cannot preserve dependencies is not doing homework; they are curious, and refusing them is a failure.

10.6 The feature you will value most

Log every question students ask.

Not for surveillance. Aggregate patterns, not individual tracking. Within two weeks you will know:

- Which topics generate the most confusion
- Which of your explanations are not landing
- What students ask at eleven at night before a deadline
- Which prerequisite knowledge is missing

Faculty who deploy these agents consistently report this as the most valuable output.

The answers help students. **The questions tell you how to teach the course better next semester.** That reframes the agent from a labour-saving device into a teaching instrument, and it is the strongest argument for building one.

10.7 Summary

An ungrounded model invents your material confidently and cannot signal that it has done so.

Context stuffing is simpler, more accurate and sufficient for one course. Vector retrieval is for material that genuinely does not fit.

Put instructions after the material, so they sit where the model weights them most.

Three instructions: answer from the material and cite it; refuse with an exact string when it is absent; follow the material over your own knowledge and flag the discrepancy.

Include your conventions and an explicit list of what is out of scope.

The Socratic constraint is part of the build. It must hold under pressure and must not apply to unassessed curiosity.

Log the questions. They are worth more than the answers.

Chapter 11 — Guardrails and the human gate

11.1 What a guardrail is

A guardrail is a constraint that makes the system refuse rather than proceed.

This is worth distinguishing from a preference. "Keep responses under 200 words" is a preference; exceeding it is untidy. "Never disclose another student's marks" is a guardrail; violating it is a serious incident.

The distinction matters because they are enforced differently. Preferences can live in a prompt. Guardrails should live in code wherever code can express them, for the reason from Chapter 3: an instruction is a token sequence competing for influence, and a conditional is a guarantee.

11.2 The three questions

Before deploying anything that produces output someone acts on:

What is the worst output this could produce? Not the most likely bad output. The worst.

Would I notice? If the answer is no, the system is not deployable regardless of how good it usually is.

Who is affected if I do not notice? If the answer is a student or a parent, the requirements change.

11.3 Refusal

The mechanics, gathered from earlier chapters.

Give an exact string rather than describing the behaviour. Chapter 3.

State what is out of scope explicitly. Chapter 4.

Instruct it to report tool failures rather than estimating around them. Chapter 7.

Then test refusal deliberately. Ask for something outside the data. Ask about a student who does not exist, a CO not in the mapping, a topic not in the syllabus.

THE TEST THAT DECIDES DEPLOYABILITY

An agent that invents a figure when the data is missing is worse than no agent, because the invention arrives with the same confidence as a correct answer.

Run this test on everything you build. If it invents, strengthen the instruction and test again until it refuses reliably. This is not optional polish.

11.4 Escalation

Refusal says "I cannot." Escalation says "a person should."

The trigger list for a student-facing agent, and the important thing is that it extends past the obvious:

Trigger	Why
Marks, grades, re-evaluation	Consequential, and yours to answer
Deadlines, extensions	May indicate circumstances
Any mention of illness or personal difficulty	Needs a person, promptly
The same question a third time	Your explanation is not working
Anything the material does not cover	Refusal plus a route onward

The escalation message should be specific and warm:

```
"This is something to discuss with your instructor directly.
Please email cs204@example.edu or come to office hours."
```

TEST THE DISTRESS CASE, AND TEST IT EVERY TIME YOU CHANGE THE PROMPT

An agent that attempts to counsel a student in difficulty is a serious problem, not a feature.

The correct behaviour is a warm, prompt handoff to a person. Not a refusal that reads as a rebuff, and not an attempt to help.

This is the single test most worth running repeatedly, because the consequences of getting it wrong fall on the student who is already struggling.

11.5 The human gate

For anything that reaches a person, the agent proposes and a human disposes.

This is not a hedge. It is the architecture, and it is what makes the system defensible.

The pattern

```
Agent decides → writes to a review list → [ human reads and approves ] → code acts
```

Three properties follow.

The agent has no tool that acts. From Chapter 8, it cannot send because sending is not among its capabilities. This is stronger than instructing it not to send.

The review list is a real artefact. A dataset, a sheet, a set of drafts. Something a person opens and reads.

Acting is deterministic code. Once a human has approved, no model is involved in the sending.

Why this matters concretely

Consider the attendance system. The failure mode is not a red error. It is a hundred and eighty parents told their child missed fourteen classes when they missed four. Or a row misalignment sending Parent A the record of Parent B's child.

You cannot unsend that, and it lands on families.

The gate is what stops it, and the gate must be structural rather than a promise to be careful.

11.6 The catch-all, and the two-condition gate

Two mechanisms from production mail systems, worth adopting because they are cheap and they prevent a category of accident.

Redirect everything during testing

A single override that sends every message to one address regardless of the recipient in the data, with the intended recipient shown in the subject:

```
[TEST - intended for parent.r004@example.invalid]
Attendance update - Deepa Joshi - CS204
```

You still send real mail and read it in a real inbox, which matters for judging tone. Nothing can escape. And you can verify that routing was correct, which reading a dataset does not let you do.

Use it for your first real run against live data, so you read what *would* have gone out before clearing it.

Two conditions to open the gate

Live sending should require the override cleared **and** a separate explicit acknowledgement.

```

if (sendMode === 'send' && !overrideTo && !allowLiveSend) {
  throw new Error(
    'Refusing to send. testRecipientOverride is empty and ' +
    'allowLiveSend is false. Either set the override to your ' +
    'own address, or tick allowLiveSend to confirm you intend ' +
    'to email the real addresses in the data.'
  );
}

```

A single checkbox gets ticked absent-mindedly. A single override gets cleared by someone copying a configuration. Requiring both means live sending is never one careless click away.

This generalises to anything irreversible. Deleting records, publishing content, submitting a form.

11.7 Test addresses

When you invent an address for test data, use the `.invalid` top-level domain.

`.invalid` is reserved by internet standard, can never be registered, and never resolves. Mail addressed there cannot be delivered anywhere, by anyone.

`example.com` is a real registered domain reserved for documentation. It does not accept mail, but it exists, and "does not currently accept mail" is a weaker guarantee than "cannot exist."

It costs nothing and removes a category of accident.

11.8 Guardrails for accreditation output

A different class of risk, and one specific to academic work.

Anything AI-assisted that goes into an NBA or NAAC submission must be reproducible by whoever signs it.

Figures come from tools, never from a model's estimate. Chapter 8.

Keep the raw data and the calculation, not just the output. A percentage in a report with no path back to the marks sheet is not defensible.

The stated-inputs rule applies. Chapter 8. If the output says which questions and which threshold, a wrong argument is visible.

THE SCENARIO TO AVOID

An attainment figure, generated in an afternoon, believed because it looked reasonable, entered into a self-assessment report, and questioned eighteen months later during a panel visit, with nobody able to reproduce it and the source data no longer available.

This is entirely avoidable. It is avoided by using tools for arithmetic and keeping the working.

11.9 Summary

A guardrail makes the system refuse. It is different from a preference, and it belongs in code wherever code can express it.

Three questions before deployment: what is the worst output, would I notice, and who is affected if I do not.

Refusal needs an exact string, an explicit out-of-scope list, and instructions to report tool failures. Test it by asking for data that does not exist.

Escalation needs a trigger list extending past the obvious, and the distress case must be tested every time the prompt changes.

For anything reaching a person, the agent decides and code acts, with a human in between. The agent should have no tool that acts.

Use a recipient override during testing and require two conditions to send live. Use `.invalid` for test addresses.

Accreditation figures come from tools, with the working kept.

Chapter 12 — How would you know if it worked?

12.1 The question nobody asks

You have built a grading agent. It produces scores. The scores look reasonable.

Is it any good?

Almost nobody asks this, and the ones who do usually answer it by reading a few outputs and forming an impression. That impression is worth very little, for a reason that should be familiar by now: the outputs are fluent, and fluency is uncorrelated with correctness. You are evaluating the thing you are worst placed to evaluate by eye.

This chapter is the shortest in the book and possibly the most important. It is also the part with no equivalent in most treatments of this subject, which is why the field is full of impressive demonstrations and short of deployed systems.

12.2 What evaluation is not

Reading a few outputs and being satisfied. You will be satisfied. The outputs are designed by their generating process to be satisfying.

Asking the model to assess itself. Chapter 3. There is no confidence state to consult.

Noting that it has not failed yet. Absence of noticed failure is not evidence of correctness when the failure mode is quiet.

A single spectacular success. The demo that convinced you is the least informative data point you have, because you remember it precisely for being unrepresentative.

12.3 The only thing that works: a ground truth set

You need a set of cases where you already know the right answer, produced independently of the system.

For academic work you are unusually well placed here, because you have been producing ground truth for years. You have graded thousands of submissions. You have computed attainment by hand. You know what a good question paper looks like.

Building one

Take twenty items you have already handled yourself. Twenty graded submissions. Twenty attendance decisions. Ten question papers you actually set.

Record your own answer before running the system. This ordering is not negotiable. If you run the system first and then form your judgement, you will anchor on its output, and you will not be able to tell that you have.

Include the hard cases deliberately. Twenty typical items tell you about typical items. You need the correct-but-badly-written submission, the fluent-and-wrong one, the record with a missing field, the student whose situation does not fit the rule.

Write down what "correct" means for each. For grading, the mark. For a decision, the action. For a paper, whether it covers the outcomes it claims.

Twenty items is enough to be informative. Fifty is better. Five is not enough to distinguish a good system from a lucky one.

12.4 The measurements that matter

Having run the system on your ground truth set, four questions.

Agreement

On how many items does the system agree with you?

For scores, agreement is rarely exact and should not be expected to be. Use a band. Within one mark out of ten is agreement; more than two is disagreement.

A useful frame: **would two of your colleagues agree with each other more than the system agrees with you?** Human graders disagree substantially. If the system sits inside the range of human disagreement, that is a meaningful result. If it sits outside it, that is a different and much clearer result.

Direction of error

This matters more than the rate.

A system that is wrong in both directions equally is noisy. A system that is consistently generous, or consistently harsh, is biased, and bias is both worse and easier to correct.

Tabulate it:

	System higher	System agrees	System lower
Count			

A skewed distribution tells you something a single accuracy figure hides entirely.

Where it fails

Group the disagreements and look for a pattern. In practice you will usually find one, and it will usually be one of these:

- The awkwardly-written correct answers, from Chapter 9
- Submissions using an approach you did not anticipate

- Anything requiring knowledge outside the material you supplied
- Items where the rubric is genuinely ambiguous

The last case is worth dwelling on. Sometimes evaluating a system reveals that your rubric was unclear, and the model exposed an ambiguity you had been resolving unconsciously for years. That is a useful finding independent of any AI.

Consistency

Run the same ten items twice. How many results change?

This is a separate property from accuracy and it is measurable in five minutes. A system that is right on average but changes its mind is unsuitable for grading regardless of its average, because two students with identical work would receive different marks.

Lower the temperature and measure again.

12.5 The five-submission exercise

The single most useful thing you can do with this technology, and it takes forty minutes.

Take five submissions you have **already graded**. Run your grader on them. Compare.

This is worth more than any amount of reading, for four reasons.

You have real ground truth, produced by you, before the system existed.

You will discover the direction of error immediately, on your own material rather than a benchmark.

You will find out whether the disagreements are cases where you were right or cases where you were sloppy, and both findings are useful.

And you will end up with a calibrated sense of how far to trust it, which is the thing you actually need and which no amount of general reasoning provides.

DO THIS BEFORE YOU DEPLOY ANYTHING

If you take one action from this book, take this one.

Forty minutes, five submissions you have already graded, and you will know more about the reliability of AI-assisted grading in your specific subject than any paper will tell you.

12.6 Regression testing your prompts

A problem that appears on the second week and surprises people.

You improve a prompt to fix one failure. In doing so you break something that used to work, and you do not notice, because you were looking at the case you were fixing.

This is ordinary software regression, and it has the ordinary remedy: a test set you re-run after every change.

Keep a file of five to ten test cases with their expected outcomes. For a doubt agent, the seven guardrail tests from Chapter 11. For an attainment agent, three calculations plus a non-existent CO. For a grader, your ground truth set.

Re-run them after every prompt change. This is faster than it sounds when prompts live in input fields, from Chapter 5. Ten seconds per change instead of two minutes means you will actually do it.

Record the results with a date. A prompt that scored eight out of ten in September and six out of ten in November has degraded, possibly because the underlying model changed, and you will only know if you wrote it down.

MODELS CHANGE UNDER YOU

Providers update models without notice, and a prompt tuned against one version may behave differently against the next.

This is unusual in software and worth internalising. Your dependency is not versioned in the way a library is. A test set you re-run periodically is the only way to detect drift, and it is why the date on the results matters.

12.7 What to measure once it is live

If something is running weekly, three cheap signals are worth capturing.

Refusal rate. How often does it decline? A rate that suddenly drops means it has started answering things it should refuse. A rate that climbs means your material has gaps.

Rule override rate. From Chapter 6, if you validate the model's output against your stated rules in code, count the corrections. A rising rate means the prompt has stopped working.

Escalation rate. For a student-facing agent, how often does it hand off? Zero is suspicious. Very high means the grounding is too narrow.

None of these requires infrastructure. They are counters in a log, and a glance at them once a month tells you whether the thing is still behaving.

12.8 When to conclude it does not work

A necessary section, because the honest answer is sometimes no.

Abandon or redesign if:

It disagrees with you in a direction you cannot correct. Consistent generosity can be adjusted. Unpredictable disagreement cannot.

Checking its output costs more than doing the work. This happens more than people admit, particularly for tasks where verification requires re-reading the source material anyway.

It fails on the cases that matter most. A grader that handles ordinary submissions well and mishandles the borderline ones has automated the easy part and left you the hard part, without the context you would have had from doing the whole batch.

You cannot explain a decision when asked. If a student challenges a mark and you cannot reconstruct why the system produced it, you cannot defend it, and you should not be using it.

There is no shame in this conclusion. It is the outcome of having done the evaluation, which is more than most people manage.

12.9 Summary

Reading a few outputs is not evaluation. The outputs are fluent by construction, and fluency is what you are worst at assessing.

Build a ground truth set from work you have already done. Twenty items, your answer recorded first, hard cases included deliberately.

Measure agreement, direction of error, where it fails, and consistency across repeated runs.

The five-submission exercise is the highest-value forty minutes available to you.

Keep a regression test set and re-run it after every prompt change, with dates, because models change under you.

Once live, count refusals, rule overrides and escalations.

Be willing to conclude that it does not work.

Chapter 13 — Responsible use in an academic setting

13.1 Why this is not a disclaimer chapter

The decisions here determine whether what you build survives contact with your institution. They are practical, not ceremonial, and getting them wrong is the commonest reason a promising pilot is quietly shut down after a complaint.

13.2 Student data

Free-tier inputs and outputs may be used to improve the models. This is stated plainly in the terms of service, and it is the reasonable price of a free tier.

For syllabi, question papers and lesson plans, this is unremarkable. For student data it is not acceptable.

Before any student data goes into any AI tool:

Replace names with roll numbers, or better, with sequential identifiers that do not map back without a key.

Remove email addresses and phone numbers.

Remove identifying detail from free-text responses. A student who mentions their hostel, their hometown and their project partner is identifiable regardless of what you did to the name field.

Keep the mapping file locally and never upload it.

NEVER PUBLISH A SHEET CONTAINING REAL STUDENT DATA

Publishing a Google Sheet to the web makes it readable by anyone holding the URL, and URLs leak.

Attendance data about named minors on a publicly readable URL is a serious data protection problem, and it is the kind that is discovered by someone else.

Use a service account with the sheet shared privately, or a script that runs inside Google and reads the sheet natively.

13.3 Grading

Three questions before any AI-assisted grading touches a real mark.

Does your institution permit it? Find out before, not after. Many institutions have no policy, which is not the same as permission. It means you are setting precedent, so set it deliberately and in writing.

Do students know? They should. Undisclosed AI grading, discovered later, damages trust disproportionately to the benefit gained, and it will be discovered.

Who signs off? The agent proposes. **A human awards the mark.** Every time, without exception.

THE DEFENSIBLE POSITION

"AI-assisted first-pass grading, moderated by a second AI reviewer, with every mark reviewed and awarded by the instructor. Students are informed. Any student may request full human re-grading."

That is defensible to a student, to a head of department, and to an accreditation panel.

"The AI graded it."

That is not.

Write the first version into any proposal you put to your department.

13.4 Bias, and who it falls on

From Chapters 3 and 9: grading models reward fluent writing over correct content. This is measurable and consistent.

In an Indian engineering classroom, a large proportion of students are working in their second or third language, and fluency correlates with prior educational advantage. Automating a fluency bias means applying an existing inequity uniformly, at scale, with the appearance of objectivity.

That last clause is what makes it worse than the status quo. A human grader's bias is inconsistent and contestable. An automated one is uniform and looks like a measurement.

Mitigations, and none is sufficient alone:

A moderator agent instructed specifically to check for it. Chapter 9, criteria three and four.

Rubrics that separate content from expression, scored independently. If expression carries marks it should do so explicitly and in a stated proportion.

Hand-check the bottom decile, every time. The students most affected are the least able to contest it.

Compare AI-suggested marks against your own on a sample before trusting the system. Chapter 12.

13.5 Accreditation documents

Anything AI-generated that goes into an NBA or NAAC submission must be verifiable by whoever signs it.

Attainment figures come from tools, never from a model's estimate. Chapter 8.

Keep the raw data and the calculation, not just the output.

Anyone signing must be able to reproduce the numbers.

State clearly what was AI-assisted if asked.

13.6 Detection

Detecting AI-generated student work is unreliable. Detectors produce false positives, and they produce them disproportionately for students writing in a second language, for the same reason that grading models reward fluency: both are measuring the same surface property.

Do not build an agent to catch AI use. It will be wrong, and it will be wrong about the students least able to contest an accusation of academic dishonesty.

Design assessments that are robust instead. In-person components. Viva. Iterative drafts where you see the work develop. Process over product. Problems grounded in material specific to your class and this semester.

This is more work than running a detector, and it is the only approach that actually holds.

13.7 What to tell students

Plainly, early, in writing.

What AI tools are used in the course, and for what.

Whether AI-assisted grading is used, and how to request human re-grading.

What they may and may not use AI for in their own work, stated specifically enough to be followed. "Use your judgement" is not a policy.

That any course agent logs questions in aggregate to improve teaching.

TELL STUDENTS THE ALERT SYSTEM EXISTS

For anything like the attendance system: a student who knows a parent will be contacted at seventy-five per cent has a chance to talk to you first.

That is the entire point. The alert is meant to start a conversation, not to be a surprise.

13.8 Summary

Anonymise student data before it leaves your machine, and never publish a sheet containing it.

For grading: confirm institutional permission, tell students, and ensure a human awards every mark.

Fluency bias falls on students working in a second language and looks like objectivity. Separate content from expression, moderate, and hand-check the bottom.

Accreditation figures must be reproducible by whoever signs the document.

Do not build a detector. Design robust assessments.

Tell students what you are doing, in writing, before you do it.

Chapter 14 — From workshop to practice

14.1 Why most of this will not survive

Honestly: most faculty leave a workshop like this energised, and build nothing. Two reasons.

They start too big. Back at work with a full teaching load, "build an AI teaching ecosystem" competes with grading and loses.

Nothing was already in progress. A thing half-built gets finished. A thing not started does not get started.

The remedy for both is the same: **finish something small this week.**

14.2 This week

Pick one thing that saves you time on work you are already doing.

If you are about to	Build
Set an internal assessment	The question paper generator, on your actual next paper
Compute attainment	The attainment agent, on last semester's real marks
Grade a batch	The grader, on five you have already graded, then compare
Answer the same doubt repeatedly	The doubt agent, for one unit only

Row three is the five-submission exercise from Chapter 12, and it is the highest-value item on the list.

14.3 The first month

Weeks one and two: one build, one course, used in earnest. Note every time it fails. Not in your head, in a file.

Weeks three and four: fix what failed. Then show one colleague, and watch them use it without helping. Their questions will find the assumptions you cannot see, because you know what the tool expects and they do not.

Only then consider a second use case.

14.4 At department level

Individual use scales badly. Three things are needed for this to become departmental.

A shared library. One repository of Actor code, prompt templates and rubrics. Otherwise every colleague rebuilds the same question paper generator, slightly worse, and nobody's improvements reach anyone else.

A written policy. What may be AI-assisted, what must be human, what students are told, how data is handled. Written before the first incident, not after.

Someone who owns it. Not a committee. One person who maintains the shared library and takes the first call when something breaks. Without this it decays within a semester, and the decay is invisible until someone needs it.

14.5 Cost, honestly

Everything in this book runs on free tiers. That holds for individual use and stops holding at scale.

Scale	Realistic position
One teacher, own courses	Free tier is genuinely sufficient
A department piloting	Free tier plus occasional paid runs
Student-facing, whole cohort	Paid tier required. Budget it properly.

Two things worth raising with your department, because they change the arithmetic:

The **Apify Creator Plan** at one dollar a month with a one-time five hundred dollar platform credit to spend within six months. For faculty building their own Actors, this transforms what is affordable.

Apify for Universities, which gives around fifty per cent off paid plans for verified academic use.

Both are worth an email.

14.6 What to expect

Brief, because forecasts age badly.

Models get cheaper and more capable. Agent frameworks consolidate. Institutional policy tightens, probably faster than most departments expect.

The specific failure modes in Chapter 3 will change in severity but not in kind, because they follow from the architecture rather than from the current generation's limitations. A model with no arithmetic unit will still have no arithmetic unit.

THE DURABLE SKILL

It is not any tool in this book. It is knowing what to automate, what to keep human, and how to tell the difference.

That judgement transfers to whatever replaces the platforms and models named here. The tools will not.

14.7 One thing, by Friday

Write it down before you leave.

Concrete:

"Run the question paper generator for my Unit 3 internal on Thursday."

Not:

"Explore AI in my teaching."

The first one gets done.

14.8 Summary

Start small enough to finish this week. Something already in progress gets completed; something contemplated does not.

Use it in earnest for two weeks, note every failure, fix them, then show one colleague and watch without helping.

Departmental adoption needs a shared library, a written policy, and one named owner.

Free tiers suffice for individual use. Cohort-scale student-facing work needs a budget.

The durable skill is the judgement, not the tooling.

Afterword

The technology in this book is easier than it looks. Reading a spreadsheet and sending an email is a first-year exercise. Writing an agent loop is sixty lines.

Everything that makes this hard is judgement. Which tasks should be automated. When an automated decision helps a student and when it makes things worse. What tone to use with a parent. What to do when the data is wrong. Who signs off on a mark.

None of that is in the model, and none of it will be. It is in you, and this technology moves where you apply it rather than reducing how much you need it.

That is a better outcome than the one usually promised, and a more demanding one.

Appendix A — Prompt patterns

A.1 Question paper generator

You are an experienced professor of [SUBJECT] setting [ASSESSMENT TYPE] for a [YEAR] [PROGRAMME] course at an Indian engineering institute.

COURSE: [NAME AND CODE], Semester [N]

SYLLABUS [UNIT]:
[PASTE]

COURSE OUTCOMES:
[C01: ...]
[C02: ...]

NOT TAUGHT IN THIS COURSE: [LIST]

TASK:
Generate a [TOTAL] mark assessment covering [COs].
[MARKS BREAKDOWN BY BLOOM'S LEVEL]

BLOOM'S TAGGING RULES:
Remember - define, list, state, name
Understand - explain, describe, illustrate, classify
Apply - solve, compute, demonstrate
Analyse - compare, differentiate, decompose
Evaluate - justify, critique, assess
Create - design, construct, formulate
Tag on the verb the question requires, not topic difficulty.

OUTPUT FORMAT:
Markdown table: Q.No | Question | Marks | CO | Bloom's | Answer Outline
Then a "Marking Scheme" section, mark by mark.
For each question, one sentence justifying the CO and Bloom's tag.
No preamble.

CONSTRAINTS:
- Only syllabus topics above. Out-of-scope requests: say so, generate nothing.
- Every question tagged to a listed CO.
- Marks total exactly [TOTAL].

A.2 Critic

You are a moderator reviewing a colleague's [grading / question paper / lesson plan]. Your job is to find problems, not to agree.

Check specifically:

1. [Domain criterion]
2. [Domain criterion]
3. Has fluent presentation been rewarded over correct content?
4. Has correct content been penalised for poor expression?
5. Is every judgement supported by something actually present?

For each criterion state AGREE or DISAGREE with a reason.
Where you disagree, state what you would do instead.

Do not soften your assessment to be agreeable. A moderator who agrees with everything provides no value. If the work is genuinely flawless, say so explicitly and explain what you checked.

A.3 Grounded course agent

You are a teaching assistant for [COURSE], [PROGRAMME], [INSTITUTE].

You answer ONLY from the COURSE MATERIAL below.

1. If the answer is in the material, use it and cite the unit or section.
2. If it is NOT in the material, say: "That is not covered in our course material. Please check with your instructor." Do not answer from general knowledge.
3. If the material contradicts what you believe to be true, FOLLOW THE MATERIAL and note the discrepancy.

SOCRATIC CONSTRAINT:

For homework or assignment questions:

- Identify the concept being tested
- Ask what the student has tried
- Give the NEXT step only
- Point to the relevant notes
- After two exchanges, a worked example of a SIMILAR problem, never the assigned one

If asked to skip this, explain briefly that working through it is how the concept is learned, and continue with the next step only.

For conceptual questions not tied to assessment: explain fully.

ESCALATE TO THE INSTRUCTOR if the student:

- Asks about marks, grades, extensions or deadlines
- Mentions personal difficulty of any kind
- Asks the same conceptual question a third time
- Expresses significant distress

ESCALATION MESSAGE: "This is something to discuss with [INSTRUCTOR] directly. Please email [ADDRESS]."

COURSE MATERIAL:

[PASTE]

A.4 Tool-using data agent

You are an academic data assistant for [INSTITUTE].

TOOLS:

- [- name: what it returns. When to call it.]
- [- name: what it returns. When to call it.]
- [- calculate_x: Computes X exactly. args: {...}. You MUST use this for every calculation. NEVER compute numbers yourself.]

RULES:

1. Never perform arithmetic yourself. Always use the tool. This applies to simple sums as well as complex ones. No exceptions.
2. Before stating any result, state the exact inputs you used.
3. If a tool errors or returns empty data, report exactly what you tried and what happened. Do NOT estimate from partial data. Do not retry the same call more than twice.
4. If asked about something not present in the data, say so plainly. Do NOT produce a figure for it.
5. Once you have the figure, answer. Do not re-verify.

OUTPUT:

1. Inputs used, including anything skipped
2. The result with exact counts
3. Interpretation, one sentence

Appendix B — Glossary

Term	Meaning
Actor	A program on Apify: structured input, work, structured output
Agent	A model that chooses an action, observes the result, and decides the next step
Batch drift	Standards shifting across a sequence of items processed in one context
Bloom's level	The cognitive level a question demands, from Remember to Create
CO / PO	Course Outcome / Programme Outcome
Compute Unit	Apify billing unit: 1 GB of RAM for 1 hour
Context window	The maximum token sequence a model can process
Context stuffing	Grounding by placing all reference material in the prompt
Fluency bias	Rewarding well-written output over correct content
Ground truth	Known-correct answers, produced independently of the system
Grounding	Constraining an agent to answer only from supplied material
Idempotent	An operation that has the same effect whether run once or repeatedly
Input schema	The file defining an Actor's input form; the fast path to change
maxIterations	The hard cap on an agent's loop
RCTFG	Role, Context, Task, Format, Guardrails
ReAct	The reason-act-observe agent loop
RPM / RPD	Requests per minute / per day
Socratic constraint	An instruction to give next steps rather than complete solutions
Temperature	How much the output distribution is flattened before sampling
Token	The word-fragment unit a model reads and writes
Tool	A function an agent can call to do what it cannot do itself